DATA ENGINEERING MASTER PREP • EYE-CARE WARM PARCHMENT EDITION

Pillar 2: Distributed Systems, Apache Spark & Kafka — First-Principles Guide

Apache Spark Execution, Joins & Skew Optimization • Apache Kafka Internals, Replication & EOS

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

8 Sections Indexed

Day 3: Apache Spark Execution, Joins & Skew Optimization ..... p. 2

1. Spark Execution Hierarchy: Jobs, Stages & Tasks (DE07) ..... p. 2
2. Physical Join Strategies in Spark SQL (DE08) ..... p. 2
3. Diagnosing & Fixing Partition Data Skew (DE09) ..... p. 3

Day 4: Apache Kafka Internals, Replication & EOS ..... p. 4

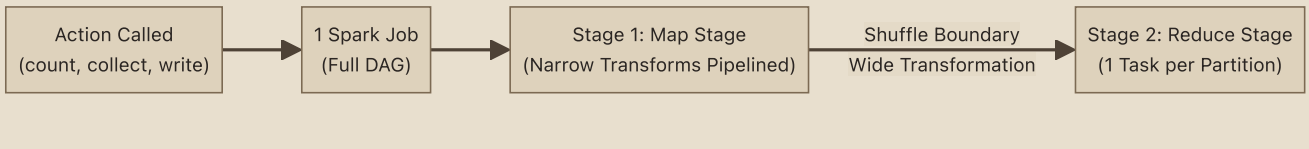
1. Partitions, Consumer Groups & Parallelism Invariants (DE10) ..... p. 4
2. Broker Replication, ISR & The Zero-Data-Loss Triad (DE11) ..... p. 4
3. Delivery Guarantees & End-to-End Exactly-Once Semantics (DE12) ..... p. 5

Day 3: Apache Spark Execution, Joins & Skew Optimization

↑ Mục Lục

1. Spark Execution Hierarchy: Jobs, Stages & Tasks (`DE07`)

Apache Spark's Catalyst Optimizer and DAG Scheduler compile high-level DataFrame / SQL queries into an execution plan:



- **Lazy Evaluation:** Transformations (`select` , `filter` , `join` , `groupBy`) only build the logical/physical plan. Nothing executes until an **Action** (`count()` , `collect()` , `show()` , `write.parquet()`) is invoked.
- **Narrow vs. Wide Transformations:**
  - **Narrow Transformation (0 Network Shuffle):** Each output partition is computed from at most one input partition (or colocated partitions on the same executor). Examples: `select` , `filter` , `withColumn` , `mapPartitions` , `union` , `coalesce(smaller_N, shuffle=False)` , and **Broadcast Hash Join**. Consecutive narrow transformations are fused into a single in-memory pipeline (**Whole-Stage CodeGen**) within **the same Stage**.
  - **Wide Transformation (Shuffle = Stage Boundary):** Output partitions require records hashed/sorted from *all* parent partitions across the network. Examples: `groupBy().agg()` , `distinct()` , `repartition()` , `orderBy()` / `sort()` , and **Sort-Merge Join**. Every wide transformation introduces a **Shuffle Boundary** and starts a **new Stage**.

> ⚠️ **WARNING**
>
> **The `coalesce(1)` Upstream Parallelism Trap:** Because `coalesce(1)` (`shuffle=False`) is a Narrow transformation, placing `df.filter(...).coalesce(1)` without a shuffle boundary forces Spark to collapse the **entire Stage's upstream task count to 1 single task**, reading and filtering all input files on a single CPU core! Use `repartition(1)` if you want upstream filtering to run across 200 parallel tasks before merging into 1 output file.

2. Physical Join Strategies in Spark SQL (`DE08`)

Join Strategy	Shuffle Required?	Sort Required?	Best Use Case & Constraints
Broadcast Hash Join (`BHJ`)	**No (0 B of large table)**	No	One table fits in Driver & Executor RAM (< 10–100 MB, `autoBroadcastJoinThreshold`). Supports all joins **except** `FULL OUTER` and broadcasting the preserved side of an outer join (e.g., cannot broadcast left table in `LEFT JOIN`).

Join Strategy	Shuffle Required?	Sort Required?	Best Use Case & Constraints
Sort-Merge Join (`SMJ`)	**Yes (Both tables)**	**Yes ($O(N \log N)$)**	Default scalable join for two large tables (`500 GB` $\bowtie$ `200 GB`). Spills gracefully to disk if partitions exceed memory. Equi-joins only (`=`).
Shuffle Hash Join (`SHJ`)	**Yes (Both tables)**	No	Shuffles both tables by key, then builds an in-memory hash table of the smaller partition per task without sorting. Faster than SMJ when one side is $3\times$ smaller per partition and fits in memory.
Broadcast Nested Loop (`BNLJ`)	No	No	Fallback for **non-equi joins** (`a.ts BETWEEN b.start AND b.end`). $O(N \cdot M)$ complexity — dangerous on large tables!

---

3. Diagnosing & Fixing Partition Data Skew (`DE09`)

The Symptom: `199 / 200` Tasks Finished, `1` Straggler Stuck

When Spark performs a wide `groupBy(key)` or `join(key)`, it partitions records by:

$$\text{partition_id} = \text{hash}(\text{key}) \bmod P$$

If 40% of all rows share a single hot key (e.g., `merchant_id = NULL` or `'AMAZON'`), **every single one of those rows hashes to the exact same partition**, regardless of whether `spark.sql.shuffle.partitions` (P) is `200` or `20,000`!

Three Production Fixes for Hot-Key Skew

1. **Filter or Isolate `NULL` / Default Keys Before the Shuffle:**
   - In an `INNER JOIN`, `NULL = NULL` never matches anyway — filter `WHERE key IS NOT NULL` before joining!
   - In a `LEFT JOIN`, split `df_null = df.filter(col("key").isNull())` and `df_valid = df.filter(col("key").isNotNull())`, join only `df_valid`, and `unionByName` the two branches.
2. **Spark 3+ Adaptive Query Execution (AQE) Skew Join:**
   - Enable `spark.sql.adaptive.enabled = true` and `spark.sql.adaptive.skewJoin.enabled = true`.
   - At the end of the map stage, Spark inspects the actual byte sizes of shuffle files on disk. If Partition k exceeds `skewedPartitionFactor` ($5\times$ median) and `skewedPartitionThresholdInBytes` (256 MB), AQE splits Partition k into S sub-partitions and duplicates the matching partition from the other table.
3. **Manual Key Salting (K Buckets):**
   - **Fact / Skewed Table:** Append a uniform random integer $s \in \{0, 1, \ldots, K-1\}$ to the join key:

     $$\text{salted_key} = \text{concat}(\text{key}, \text{'_'}, \lfloor \text{rand}() \times K \rfloor)$$

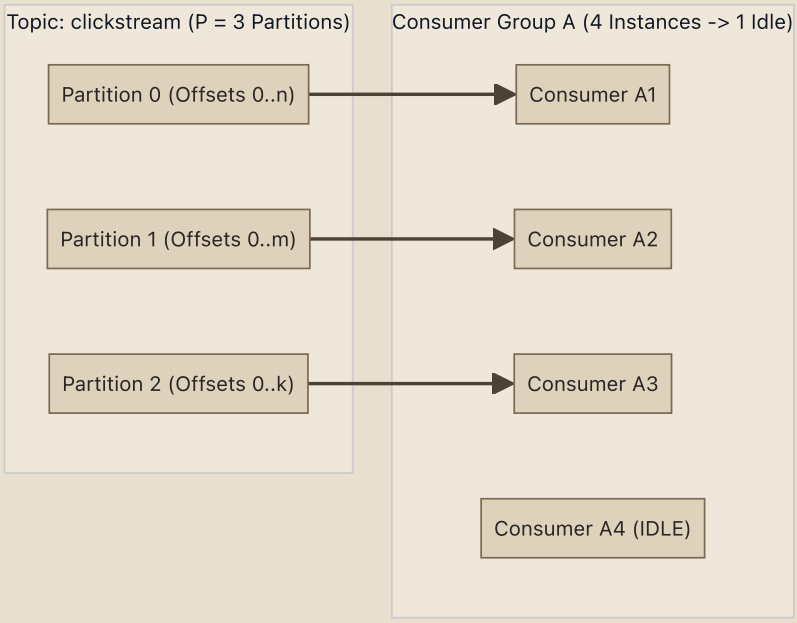
   - **Dimension / Lookup Table:** Replicate each hot key K times using `explode(array(0, 1, ..., K-1))` so every salted bucket $\{0, \ldots, K-1\}$ finds its matching dimension row across K parallel tasks!

Day 4: Apache Kafka Internals, Replication & EOS

↑ Mục Lục

1. Partitions, Consumer Groups & Parallelism Invariants (`DE10`)

An Apache Kafka **Topic** is divided into P append-only, immutable log files called **Partitions**.



1. **Key Routing & Ordering Guarantee:**
   - When a producer sends a record with a non-null `key`, Kafka routes it deterministically via:

     $$\text{partition} = \text{murmur2}(\text{key}) \bmod P$$

   - **Ordering is strictly guaranteed ONLY within a single partition** (by offset), never globally across the whole topic.
2. **Consumer Group Exclusive Assignment Rule:**
   - Within a single **Consumer Group**, each partition is assigned to **at most one** consumer instance at any time.
   - If a Consumer Group has C consumers and the topic has P partitions:
     - If $C \leq P$: All C consumers are active (some read multiple partitions).
     - If $C > P$: Exactly P consumers read 1 partition each, and $C - P$ **consumers sit completely idle** as hot standbys.
   - Multiple independent **Consumer Groups** (e.g., `realtime-fraud` and `s3-archiver`) each receive the full stream independently without interfering with each other.

---

2. Broker Replication, ISR & The Zero-Data-Loss Triad (`DE11`)

To prevent data loss when a broker crashes while maintaining high availability for writes, production Kafka clusters rely on four coupled parameters:

Parameter	Scope	Production Value	Why It Is Essential
`replication.factor` (`RF`)	Topic / Broker	`3`	Stores 1 Leader replica + 2 Follower replicas across 3 distinct racks/brokers.
`min.insync.replicas`	Topic / Broker	`2`	Requires at least 2 replicas (Leader + ≥ 1 Follower) to be in the **In-Sync Replica (ISR)** set before accepting `acks=all` writes.
`acks`	Producer	`all` (`-1`)	Producer waits until the Leader **and all current ISR followers** have written the record before marking the send successful.
`unclean.leader.election.enable`	Topic / Broker	`false`	Forbids an out-of-sync replica (not in ISR) from becoming Leader if all ISR brokers go down, preventing silent truncation of committed messages.

> ⚡ **IMPORTANT**
>
> **Why `RF = 3` and `min.insync.replicas = 2` (Not `3`):**
>
> - With `acks = 1`, if the Leader acknowledges a write and crashes 1 ms before followers pull the record, that record is permanently lost.
> - With `RF = 3`, `min.insync.replicas = 3`, and `acks = all`, if **even** 1 **broker reboots** for a rolling patch, the ISR size drops to $2 < 3$, and every producer immediately fails with `NotEnoughReplicasException` (0% write availability during maintenance!).
> - Setting `RF = 3, min.insync.replicas = 2, acks = all` guarantees every committed message lives on **at least** 2 **brokers** while tolerating 1 **broker outage** with zero downtime and zero data loss.

---

3. Delivery Guarantees & End-to-End Exactly-Once Semantics (`DE12`)

When a consumer reads a batch from Kafka, processes it, and writes to a downstream sink, the order of operations determines what happens on a crash:

1. **At-Most-Once (Commit Offset *Before* Writing to Sink):**
   - If the worker commits offset `100` to `__consumer_offsets` and crashes *before* writing to the database, upon restart it resumes at offset `101`. Message `100` is **silently lost**.
2. **At-Least-Once (Commit Offset *After* Writing to Sink):**
   - If the worker writes message `100` to the database and crashes *before* committing offset `100`, upon restart it reads message `100` again and writes it a second time (**duplicates**).
3. **Exactly-Once Semantics (EOS):**
   - **Producer-to-Broker (`enable.idempotence = true`):** Assigns each producer a `PID` and monotonically increasing sequence number per partition so broker-level network retries never create duplicate log entries.
   - **Kafka-to-Kafka (`transactional.id` + `read_committed`):** Uses Kafka's two-phase transaction coordinator (`sendOffsetsToTransaction`) to atomically commit consumer offsets and output topic

messages together.

- **Kafka-to-External Database (PostgreSQL / Snowflake / Delta):** Combine **At-Least-Once** offset commits (commit *after* DB write) with **Idempotent Sink Writes** (`INSERT ... ON CONFLICT (event_id) DO UPDATE` / `MERGE`), **or** store the Kafka `(topic, partition, offset)` inside a metadata table within the **exact same atomic ACID database transaction** as the business rows!